UNIVERSITY OF BRISTOL

School of Mathematics

**Algorithms and Programming in C(++) and R**
MATH10017
(Paper code MATH-10017W)

---

Winter 2026 2 hour 30 minutes

---

**This paper contains two sections: Section A and Section B.**
**Each section should be answered in a separate booklet.**

Section A contains FIVE questions and Section B contains TWO questions
All SEVEN answers will be used for assessment.

Calculators are not permitted.

# MOCK EXAM
## SOLUTIONS

*Do not turn over until instructed.*

## Section A: Short Questions

A1. (**8 marks**)

(a) (**2 marks**) Which one of the following statements is **wrong**?

(A) In C, when an array is passed to a function, the function receives the address of its first element.

(B) In C, a `struct` variable is passed to a function by value.

(C) Memory allocated on the heap with `malloc` is released automatically when the function that allocated it returns.

(D) R is an interpreted language, while C is a compiled language.

(E) In R, indexing a vector beyond its length returns `NA`.

**Solution**

[C. Returning from the allocating function does not release heap memory. An allocation should be released with `free` when it is no longer needed; losing track of an allocation without releasing it causes a memory leak.]

(b) (**3 marks**) Explain what is meant by **vectorisation** in R. State the **two** distinct sources of speed-up that vectorised code can provide.

**Solution**

[Answer: Vectorisation is the process of rearranging code so as to avoid explicit `for`/`while` loops in R, by using functions that operate on whole vectors or matrices at once. The two sources of speed-up are: (1) the overhead of interpreting an R loop is avoided, because the loop now takes place in compiled C code; (2) the R function being called may itself call a highly optimised compiled library (e.g. `%*%`, `rowSums`).]

(c) (**3 marks**) Let `l` be an R list. What is the difference between `l[2]` and `l[[2]]`? Give a short example to illustrate.

**Solution**

[Answer: `l[[2]]` extracts the second element itself, and it has the type of that element. `l[2]` returns a *list* of length one, containing the second element. Example:]

```
l <- list("a", 10)
l[[2]] + 1      # returns 11
class(l[2])     # returns "list"
l[2] + 1        # Error: non-numeric argument to binary operator
```

A2. (**8 marks**) Read the following C program and answer the questions.

```
#include <stdio.h>

int g(int n){
    printf("%d\n", n);  // L1
    if(n <= 1){
        return 1;
    }
    return n * g(n - 2);
}

int main(void){
    int k = 5;
    int r = g(k);
    printf("result: %d\n", r);
}
```

(a) (**3 marks**) What is the printed output of this program?

**Solution**

[Answer:]

```
5
3
1
result: 15
```

(b) (**2 marks**) Suppose we replace the line `if(n <= 1)` with `if(n == 1)` and, in `main`, we set `k = 4`. What will happen when we run the program? Briefly justify your answer.

**Solution**

[Answer: The program will not terminate normally. Starting from `n = 4`, the argument takes the values $4, 2, 0, -2, -4, \ldots$ and never equals 1, so the recursion never reaches the base case. Each call adds a frame to the stack, so the program will eventually crash with a stack overflow.]

(c) (**3 marks**) Consider the original program. Draw the stack memory layout at the moment the line marked `L1` is executed for the **third** time. Your diagram must include all local variables (including function arguments) and their values.

**Solution**

[Answer: Four boxes, from top to bottom. The variable `r` has not yet been assigned a value; it may instead be omitted with a comment explaining this.]

```
g:     n = 1
g:     n = 3
g:     n = 5
main:  k = 5, r = (not yet assigned)
```

A3. (**8 marks**)

A square matrix is said to be **upper triangular** if all the entries strictly below its main diagonal are zero. The following C code checks whether a 4 by 4 matrix stored in `A` in **column-major** order is upper triangular. It prints "1" if `A` is upper triangular and "0" otherwise.

```
#include <stdio.h>
int elem(int M[], int i, int j){
    __YOUR CODE HERE__
}

int is_upper(int A[]){
    for (int i = 0; i < 4; i++){
        for (__YOUR CODE HERE__){
            if (__YOUR CODE HERE__){
                return 0;
            }
        }
    }
    return 1;
}

int main(void){
    // A stores a 4 by 4 matrix in column-major order.
    int A[] = {1,0,0,0, 2,3,0,0, 4,5,6,0, 7,8,9,1};
    printf("%d\n", is_upper(A));
}
```

(a) (**2 marks**) Suppose that the `elem` function returns the $i, j$-th element of a 4 by 4 matrix `M` stored in column-major order. Complete `elem` using **one line of code**. Use zero-based indexing.

**Solution**

```
return M[j*4 + i];
```

(b) (**3 marks**) Complete the two missing lines in the `is_upper` function. Your inner loop should visit only the entries that actually need to be checked.

**Solution**

```
for (int j = 0; j < i; j++){
    if (elem(A, i, j) != 0){
```

[With the matrix given in `main` the program prints 1.]

(c) (**3 marks**) Suppose we use the same algorithm on an $n$ by $n$ matrix. In the worst case, how many times is the body of the inner loop executed? Express your answer as a formula in $n$, and give the time complexity of the algorithm using the $O(\cdot)$ notation.

**Solution**

[In the worst case all entries below the diagonal are zero, so row $i$ requires $i$ checks. The total is $0 + 1 + \cdots + (n-1) = n(n-1)/2$, giving time complexity $O(n^2)$.]

A4. (**8 marks**) The *running maximum* of a numeric vector `v` is the vector `out` of the same length such that `out[k]` is the largest among the first `k` elements of `v`. For example:

```
> cum_max( c(3, 1, 4, 2) )
[1] 3 3 4 4
```

Read the following R code and answer the questions. For parts (a) and (b), assume that `v` contains at least two elements.

```
cum_max <- function(v){
  n <- length(v)
  out <- numeric(n)
  out[1] <- v[1]

  for(ii in 2:n){
    # YOUR CODE HERE
  }

  return(out)
}
```

(a) **(3 marks)** Complete the function so that it works as described above.

**Solution**

```
cum_max <- function(v){
  n <- length(v)
  out <- numeric(n)
  out[1] <- v[1]

  for(ii in 2:n){
    if( v[ii] > out[ii-1] ){   # <- we added these 5 lines
      out[ii] <- v[ii]
    } else {
      out[ii] <- out[ii-1]
    }
  }
  return(out)
}
```

[The single line `out[ii] <- max(v[ii], out[ii-1])` is also worth full marks.]

(b) **(3 marks)** Explain what will be the output of this code:

```
v <- c(2, 5, 1, 7, 3)
z <- cum_max(v)
print( z[z > 2] )
```

**Solution**

[Answer: `z` is `c(2, 5, 5, 7, 7)`, so `z > 2` is `c(FALSE, TRUE, TRUE, TRUE, TRUE)` and the code prints]

```
[1] 5 5 7 7
```

(c) **(2 marks)** Using your completion from part (a), what happens if `cum_max` is called on a vector `v` containing a **single** element? Explain why.

**Solution**

[Answer: The function does not behave as intended: it stops with an error. With `n = 1` the expression `2:n` is `2:1`, which is the vector `c(2, 1)` rather than an empty vector, so the loop *does* run. At the first iteration `ii` is 2 and `v[2]` is `NA` (indexing beyond the end of a vector), so the condition of the `if` statement is `NA` and R reports an error such as "missing value where TRUE/FALSE needed".]

A5. (**8 marks**)

Let $X$ be a numeric matrix with $n \geq 1$ rows and $d \geq 1$ columns. We want a function `col_max(X)` returning a vector with $d$ elements, whose $j$-th element is the largest entry of the $j$-th column of $X$.

Answer the following questions in R.

(a) (**4 marks**) Write `col_max(X)` using at least one `for()` loop. You are **not** allowed to use the built-in functions `max()` or `apply()`.

**Solution**

```
col_max <- function(X){
  n <- nrow(X)
  d <- ncol(X)
  out <- numeric(d)
  for(j in 1:d){
    best <- X[1, j]
    for(i in 1:n){
      if( X[i, j] > best ){
        best <- X[i, j]
      }
    }
    out[j] <- best
  }
  return(out)
}
```

(b) (**2 marks**) Rewrite the function using `apply()` and a single line of code.

**Solution**

```
col_max_2 <- function(X){
  apply(X, 2, max)
}
```

[Note that the second argument must be 2 (columns), not 1 (rows).]

(c) (**2 marks**) Explain what, in your opinion, will be the output of this code:

```
X <- matrix(1:6, nrow = 2)

sum( col_max(X) > 3 )
```

**Solution**

[Answer: 2. R fills the matrix column by column, giving column maxima 2, 4 and 6. Two of these exceed 3. Summing the logical comparison counts its `TRUE` entries, giving 2.]

## Section B: Longer Questions

**Please use a new booklet for this Part.**

B1. (**30 marks**) A shop records how many units of each of its products it sells every day. The records are stored in a matrix $S \in \mathbb{N}^{m \times n}$, where $S_{i,j}$ is the number of units of product $i$ sold on day $j$. Entries may be zero.

- Suppose $m = 50$ (products) and $n = 30$ (days). Use zero-based indexing in this question.
- You are given the skeleton C code below.
- In your answers you may use the functions defined in the previous parts of the question, even if you did not answer them.

```
#include <stdio.h>

// YOUR CODE GOES HERE

int main(void){
    // S is a 50 by 30 matrix stored in row-major order.
    int S[] = {3,0,1,0,5, /*the rest of S is omitted*/};

    int i = 7;

    printf("%d\n", total_sales(S, i));
    printf("%d\n", best_day(S, i));
    printf("%d\n", n_consistent(S));
}
```

(a) (**6 marks**) Write a function `int elem(int M[], int i, int j)` which returns $M_{i,j}$, where `M` is a 50 by 30 matrix stored in row-major order.

**Solution**

```
int elem(int M[], int i, int j){
    return M[i*30 + j];
}
```

[Note that the number of *columns* (30) appears in the formula, not the number of rows.]

(b) (**8 marks**) Write a function `int total_sales(int S[], int i)` that returns the total number of units of product $i$ sold over the 30 days.

**Solution**

```
int total_sales(int S[], int i){
    int tot = 0;
    for (int j = 0; j < 30; j++){
        tot += elem(S, i, j);
    }
    return tot;
}
```

(c) (**8 marks**) Write a function `int best_day(int S[], int i)` that returns the day on which the largest number of units of product $i$ was sold. If several days are tied, returning any one of them is acceptable.

**Solution**

```
int best_day(int S[], int i){
    int best = 0;
    for (int j = 1; j < 30; j++){
        if (elem(S, i, j) > elem(S, i, best)){
            best = j;
        }
    }
    return best;
}
```

[Solutions that store the running maximum in a separate variable are equally valid. Initialising the running maximum to 0 is also fine here, because sales cannot be negative.]

(d) (**8 marks**) A product is called *consistent* if it sold **at least one unit on every one of the 30 days**. Write a function `int n_consistent(int S[])` that returns the number of consistent products.

Your function should stop examining a product as soon as it is clear that the product is not consistent.

**Solution**

```
int n_consistent(int S[]){
    int count = 0;
    for (int i = 0; i < 50; i++){
        int ok = 1;
        for (int j = 0; j < 30; j++){
            if (elem(S, i, j) == 0){
                ok = 0;
                break;   // early exit
            }
        }
        count += ok;
    }
    return count;
}
```

B2. (**30 marks**) A university stores the records of its students in an R `list`. Each element of the list is itself a list, with a tag `name` holding the name of the student, and a tag `marks` holding a numeric vector of the marks obtained by that student. For example:

```
studs <- list( list(name = "Ana",  marks = c(70, 55, 62)),
               list(name = "Ben",  marks = c(41, 58, 45)),
               list(name = "Cleo", marks = c(88, 79, 91)) )
```

Note that different students may have taken a different number of units, so their `marks` vectors may have different lengths. Assume that there is at least one student and each student has at least one mark. Treat each part independently, starting from the original list shown above.

In your answers you may use the functions defined in the previous parts of the question, even if you did not answer them.

(a) (**5 marks**) Using the list `studs` defined above, write down R expressions that:

*(i)* return the name of the second student;

*(ii)* return the second mark of the third student;

*(iii)* add to `studs` a fourth student called `"Dan"`, with marks 60 and 65.

**Solution**

```
studs[[2]]$name                 # (i)  "Ben"

studs[[3]]$marks[2]             # (ii) 79

studs[[4]] <- list(name = "Dan", marks = c(60, 65))   # (iii)
```

[`studs[[2]][["name"]]` is equally valid in (i), whereas `studs[2]$name` would not work. Another valid answer to (iii) is:]

```
studs <- c(studs,
           list(list(name = "Dan", marks = c(60, 65))))
```

(b) (**6 marks**) Write an R function `mean_mark(s)` that takes a **single** student (that is, a list with tags `name` and `marks`) and returns the average of that student's marks. You are **not** allowed to use the built-in function `mean()`.

**Solution**

```
mean_mark <- function(s){
  return( sum(s$marks) / length(s$marks) )
}
```

[So `mean_mark(studs[[1]])` returns 62.33333. A solution using a `for` loop to accumulate the sum is equally valid.]

(c) (**6 marks**) Write a function `all_means(studs)` that takes the whole list of students and returns a **numeric vector** containing the average mark of each student. Your solution should not use any explicit `for` or `while` loop, but you may use `sapply`, `lapply`, etc.

**Solution**

```
all_means <- function(studs){
  return( sapply(studs, mean_mark) )
}
```

[So `all_means(studs)` returns `c(62.33333, 48.00000, 86.00000)`.]

[ Using `unlist(lapply(studs, mean_mark))` is equally valid; using `lapply` alone is not, because it returns a list rather than a numeric vector.]

(d) (**6 marks**) Write a function `make_table(studs)` that returns a `data.frame` with two columns: a column `name` containing the students' names, and a column `avg` containing their average marks. Then write **one further line** of code which, given such a data frame `df`, returns only the rows corresponding to students whose average mark is above 60.

**Solution**

```
get_name <- function(s){
  return( s$name )
}

make_table <- function(studs){
  df <- data.frame(name = sapply(studs, get_name),
                   avg  = all_means(studs))
  return(df)
}
```

[And the further line is:]

```
df[df$avg > 60, ]
```

(e) (**7 marks**) Write a function `best_student(studs)` that returns the **name** of the student with the highest average mark. You are **not** allowed to use `which.max()`, `order()` or `sort()`. If several students tie, returning any one of their names is acceptable.

Then explain what your function returns if two students have exactly the same (highest) average mark.

**Solution**

```
best_student <- function(studs){
  avg <- all_means(studs)

  best <- 1
  for(ii in 1:length(studs)){
    if( avg[ii] > avg[best] ){
      best <- ii
    }
  }

  return( studs[[best]]$name )
}
```

[With the list given above this returns `"Cleo"`. In case of a tie, the strict inequality `>` means that the *first* of the tied students is returned.]

[A fully vectorised solution is also worth full marks, for example:]

```
best_student <- function(studs){
  df <- make_table(studs)
  return( df$name[df$avg == max(df$avg)][1] )
}
```

[The final `[1]` selects the first of the tied names. Without it, the expression would return all tied names rather than a single name.]